Player-Managed Potion Shop

Abdul Mueed (22SW058) / Abdul Aziz (22SW046)
Simulation report / 27 October 2026

Scenario and decision

You play the owner in a potion-shop management game. Before opening, you buy ingredient-and-bottle kits and brew an opening batch. One kit makes one potion. Once the shop opens, NPC customers buy from that prepared stock. A busy day can empty the shelves; a quiet day leaves money tied up in leftovers. At closing, a wholesaler buys the remaining batch at a discount.

The decision is how much gold to commit to stock before customer demand is known. Each simulated day is a separate one-day challenge. The app reports your chosen batch alongside the calculated benchmark. Ingredients and bottles are combined into the production cost; recipes and brewing time are outside the model.

Inputs and provenance

Crafting cost: 30 gold per potion. Selling price: 50 gold. Wholesale clearance: 10 gold per unsold potion. The baseline comparison uses stock quantities from 0 to 100 in steps of ten. The web form also accepts a custom minimum, maximum and step, including stock 500. The values adapt the rupee assignment on p.75 of the supplied notes; gold changes the unit label.

The demand model follows Tables 2.15 and 2.16 on p.68. Busy, Normal and Quiet profiles have probabilities 0.35, 0.45 and 0.20. Demand values are 40, 50, 60, 70, 80, 90 and 100. Their conditional percentages are Busy: 3,5,15,20,35,15,7; Normal: 10,18,40,20,8,4,0; Quiet: 44,22,16,12,6,0,0. Each row totals 100 percent.

Model conditions

One stock decision before demand is known. Fixed prices and distributions within an experiment. Independent one-day challenges, one opening batch, and no carryover after closing clearance. These conditions are declared, not presented as measured facts.

Method and worked checks

Sold = min(stock, demand)
Unsold = max(stock - demand, 0)
Unmet = max(demand - stock, 0)
Cash = selling price x sold - crafting cost x stock + recovery x unsold
Textbook score = cash - (selling price - crafting cost) x unmet

The extra unmet-demand deduction follows the supplied notes. It is an opportunity-cost scoring convention, not an additional cash payment. Both metrics are reported because they can choose different stock levels.

At stock 70: demand 50 gives cash and score 600 gold. Demand 70 gives both 1,400. Demand 90 gives cash 1,400, unmet demand 20, deduction 400 and score 1,000.

Sampling and experiment

Two independent pseudo-random draws select the day profile and conditional demand. Cumulative percentages map integer rolls 1 through 100 to outcomes. Each run shares its demand sequence across all stock quantities. Thirty runs of 1,000 shop days use seeds 20261026 through 20261055. The baseline evaluates 0,10,...,100. Recommendations apply to the comparison range selected. That gives 30,000 generated shop-day scenarios and 330,000 stock evaluations.

Exact expectation

For each demand d, P(d) sums P(profile) times P(d given profile). The seven marginal probabilities are 0.1435,0.1425,0.2645,0.1840,0.1705,0.0705,0.0245. Exact expected return sums probability times return over these outcomes, without random sampling. The app plots this benchmark beside the simulation.

Results and verification

The textbook objective selects 70 potions: simulated mean score 928.59 gold per shop day, exact expected score 931.00. At this stock, mean cash profit is 1,005.21 gold, unsold stock 9.87 and unmet demand 3.83 potions per shop day. The standard error of mean score is 2.23 gold.

Cash profit alone selects 60 potions: simulated cash mean 1,026.03 gold and exact 1,028.20. State the objective whenever presenting the recommendation.

Raising selling price to 80 gold changes the computed textbook winner to 80 potions. Its mean score is 2,734.57 against exact 2,740.10. The foregone-margin deduction automatically becomes 50 gold per unmet unit.

Thirty-one tests passed: 24 model checks and seven web checks. Checks include hand cases, the supplied ten-day textbook total, all digit mappings, an independent enumeration of 10,000 digit pairs, deterministic demand, seeds, ties, parameter changes, form validation, custom stock ranges, CSRF, result tables and trace pagination.

The original ten-day reference is checked using integer cents and reproduces USD 70.00. This verifies the underlying payoff arithmetic separately from the adapted game scenario.

Running the app

Install offline with python -m pip install --no-index --find-links vendor -r requirements.txt. Start python manage.py runserver and open http://127.0.0.1:8000/. Use python simulate.py for the reproducible baseline and python manage.py test -v 2 for tests.

Limitations and source

The inputs are illustrative classroom values. Demand remains fixed when selling price changes, so the sensitivity test measures margin effects, not customer price response. More simulated days cannot establish that these settings match a released game. Source: supplied _First three chapters.pdf, Example 2.3 pp.66-73 and rupee assignment pp.75-76. The interface and CLI use the same Python engine.